Scheduler Rankings and Performance Trade-offs in a Common-Model LLM-Serving Evaluation

SOROUSH VAHIDI, New Jersey Institute of Technology, USA

Scheduler comparisons require more than identical inputs: the execution contract determines which policy distinctions can affect request outcomes. We connect those structural consequences to measured performance in a common-model evaluation of four LLM-serving adaptations. Fixed batch timing and a residency/work bound make chunked-FCFS and decode-first equivalent; complete terminal records confirm the equality in all 1,440 matched conditions, despite possible differences in journal ordering. An inactive urgency gate leaves prompt-length admission ordering as an active mechanism. Among the three distinct behaviors, SLO-gated reduces aggregate mean latency by 11.3–34.0% against chunked-FCFS while reducing finite-window token throughput by 1.65–4.01%; BurstGPT mean window-p95 latency increases by 3.72% and 18.97%. Saved request timelines illustrate how these objectives can disagree. Source means exhibit no strict 10% practical reversals, although window-level and metric-dependent reversals occur. A post hoc analysis removing duplicate behaviors and endpoints preserves that contrast. A separate known-truth diagnostic distinguishes historical arithmetic reproduction from statistical calibration. The contribution is a verified, contract-specific explanation of which scheduling differences remain meaningful and how their rankings depend on the comparison estimand. The model isolates execution semantics; its rates do not predict a production GPU or establish a universal scheduler ranking.

CCS Concepts: • **Computing methodologies** → **Simulation evaluation**; • **Computer systems organization** → *Cloud computing*.

Additional Key Words and Phrases: LLM serving, performance evaluation, scheduling, trace-driven simulation, ranking stability

1 Introduction

A scheduler comparison assumes that different policy labels denote different ways to execute the same workload. That assumption can fail even when every policy sees identical requests. A label may select a different prefill model or resource constraint; conversely, distinct priority lists may induce the same state transitions. Establishing the comparison object therefore precedes interpreting its winners.

The historical LLM-Serving Scheduler Portability (LSSP) corpus illustrates the first problem: its implementations did not uniformly enforce prefill work and KV capacity. Reproducing their ranking arithmetic cannot separate scheduling decisions from policy-dependent execution semantics. We instead evaluate four adaptations under CM-0.1, a common contract specifying online information, legal actions, work advancement, batch timing, and resource acquisition and release. The adaptations retain selected ideas from FCFS serving, chunked prefill, decode-first scheduling, and SLAI-inspired prompt ordering [4, 6, 16].

Prior work already establishes the importance of workload choice [19], fair evaluation settings [2], and user-relevant metrics [1]. Our question is more specific: *which nominal scheduler distinctions survive a declared execution contract, and what performance comparisons remain after equivalent behaviors are identified?* The contribution is an auditable connection between the contract's state-transition consequences and the residual empirical ranking problem. It has three parts:

(1) **Structural reduction.** A residency/work bound and common batch-end timestamps make chunked-FCFS and decode-first equivalent, despite possibly different event-span ordering. Clock updates keep the urgency gate inactive; prompt-length admission and whole-prefill/decode exclusion remain active distinctions. Four labels thus yield three observed behaviors.

Author's Contact Information: Soroush Vahidi, sv96@njit.edu, Department of Computer Science, New Jersey Institute of Technology, Newark, New Jersey, USA.

(2) **Measured residual trade-offs.** Complete matched outcomes show how the surviving rules affect mean latency, tail latency, finite-window throughput, and deadline attainment. A retrospectively selected saved-record example explains their coexistence without treating it as a representative window or a causal decomposition.
(3) **Ranking interpretation.** Source-mean practical stability coexists with window-level and metric-dependent reversals. A post hoc view removing duplicate behaviors and endpoints checks that this contrast is not an artifact of the nominal comparison count.

The deliberately fixed batch cost removes composition-dependent service costs, while bounded residency preserves admission competition. This makes CM-0.1 useful for analyzing executable distinctions, rather than predicting a particular GPU. The results neither introduce a scheduling principle nor establish that prompt-length ordering is optimal. Their scope is this contract and the complete selected corpus; completeness makes the findings checkable, not general.

Three questions organize the evidence:

RQ1: Performance. What trade-offs remain among the distinct behaviors across workload windows, arrival/deadline settings, and residency capacities?
RQ2: Rankings. How do aggregation, metric choice, exact ties, and practical margins change the resulting preferences?
RQ3: Historical interpretation. What separates reproducible historical arithmetic from calibrated inference?

The first two questions constitute the scheduler comparison. STAT01 supports the third through synthetic known-truth diagnostics; it supplies neither additional C01 performance observations nor population guarantees for its selected trace windows.

2 Related Work and Evaluation Scope

Evaluation methodology and the closest comparison. Papaioannou and Doudali study how datasets and inference scenarios change LLM-system evaluations [19]. Agrawal et al. identify anti-patterns in baseline comparisons, evaluation settings, and metric design [2]; Etalon introduces a fluidity index to capture streaming experience missed by conventional summaries [1]. These works already establish that benchmark and metric choices matter. Our study adds a different object of analysis: executable behavior under shared timing, resource, and information rules. It derives an exact equivalence for two adaptations, verifies it over the complete matched matrix, and measures the trade-offs left after that equivalence. The source-mean/window contrast then shows how aggregation changes the interpretation of those surviving comparisons. This is a contract-specific case study, not a general evaluation methodology or a new streaming metric.

Serving and scheduling mechanisms. Orca introduces iteration-level scheduling [26]; vLLM's PagedAttention addresses KV fragmentation and sharing [16]; Sarathi-Serve combines chunked prefill with stall-free batching [4]. Fu et al. learn relative output-length order to approximate shortest-job-first scheduling [12]. Our prompt-length ordering uses observed inputs, without predicting output length or claiming the same objective or optimality. Bari et al. distinguish the throughput-optimal RAD construction from practical SLAI, which combines decode urgency and known prompt lengths [6]. SOLA jointly considers request/system state and work allocation to improve SLO attainment [15]. Their active state-dependent mechanisms differ from the fixed, inactive urgency gate. The published systems establish the value of their mechanisms under their own execution assumptions; they do not establish this adaptation panel's state equivalence. Full-context reservations also omit native paging, while disaggregation and migration, exemplified by DistServe and Llumnix [22, 28], lie outside the evaluated contract.

Binding constraints and throughput. Dai et al.'s May 18, 2026 revision studies queueing stability with token-load-dependent batch time and a maximal-request-batch-size extension [11]. The binding constraint matters in both studies, but the estimands differ. Our contract fixes batch time, limits residency to four or eight requests, and measures output divided by finite-window completion duration. Its chunked work cannot exhaust the token budget. These drained rates are not a stability region, and the observed throughput losses neither refute Dai et al.'s results nor establish a new optimality theorem.

Workloads and simulators. BurstGPT and ServeGen characterize production-derived workloads [24, 25]; resampling has a longer history in scheduler evaluation [27]. Our analysis instead conditions on a fixed selected corpus, with equal-window source means and explicit matched comparisons. Vidur combines profiling and prediction [3]; LLMServingSim and its heterogeneous/disaggregated successor model hardware/software interactions [9, 10]. They pursue hardware-aware performance estimation. Our fixed-duration work ledger pursues checkable execution semantics. Following simulation verification/validation terminology [20], journal reconstruction verifies conformance to that model; physical prediction would require separate validation.

Inference and evidence reuse. STAT01 applies established BH, BY, Holm, and concentration methods [7, 8, 13, 14]. Multiplicity correction does not validate input p-values or turn a zero-effect test into a practical-effect test. The diagnostic investigates those distinctions under specified synthetic truth. This study reuses historical windows, reconstruction, and archived rule outputs, with separate execution identities for the common-model comparison (Appendix B). Shared infrastructure and related mechanism studies do not provide independent validation of its results.

3 A Common Execution Contract

3.1 Why the comparison object matters

The historical corpus contains 9,360 configurations and two deterministic verification passes per configuration. Its primary panel contains eleven policies, with two additional style approximations. The historical engine permits policy-dependent service modes: instantaneous prefill for some policies, explicit prefill work for others, and differing enforcement of KV limits. A policy's requested work can also differ from the engine's realized progress. Consequently, those outcomes cannot be interpreted as a controlled comparison of scheduling order. Table 1 distinguishes the three evidence objects used in this paper.

The positive modeling question is whether admission and prefill/decode rules still produce different completion distributions when every adaptation faces the same online information and resource constraints. Fixing batch cost removes composition-dependent execution cost from this comparison; retaining four or eight resident slots preserves competition for admission. This makes it possible to distinguish a change in the order of work that all executes from a change in which requests can enter or progress. These are properties of the implemented abstraction. We do not claim that its settings were selected in advance to produce the equivalence or trade-offs found here.

3.2 Service, resources, and information

CM-0.1 represents one abstract device. Each batch lasts 1 ms and has a 32,768-token work budget. Each executed prompt token consumes one unit; each selected decode request consumes one unit and emits one token. All policies share these rules. A request emits no output during its own prefill, including its final chunk; its decoding starts in a later batch.

Table 1. Evidence objects and the conclusions they support. Historical identifiers containing "faithful" are immutable names, not validation certificates.

Object	Comparison or check	Interpretation
Historical LSSP	Policy-dependent service/resource modes; archived ranking arithmetic	Motivation and historical-rule counts
C01-FULL	Four adaptations; common CM-0.1 execution and information	Descriptive finite-corpus performance
STAT01	Synthetic independent studies with known population truth	Scenario-specific inference calibration

Other resident requests may decode in the same batch. Batch duration is independent of its composition. Thus seconds and tokens per second below are model quantities, not estimates for a particular GPU or LLM.

Each resident request reserves the complete 32,768-token context bound. All frozen request lengths fit; the maximum prompt-plus-output length is 30,053. Capacities of 131,072 and 262,144 tokens therefore permit *four and eight resident requests*, respectively, even though the configured sequence limit is 64. Actual used KV follows executed prompt and output prefixes, rounded to 16-token blocks, and must remain within the reservation. Completion releases the reservation. Waiting requests own none. This conservative rule avoids output-length prediction and allocation deadlock; it does not model incremental native paging or cache eviction.

There is no preemption, dropping, migration, prefix sharing, or recomputation. Requests arriving during a batch become eligible at its end. Idle time advances to the next arrival. The administrative horizon is the final arrival plus 50 s; a batch cannot finish beyond it. Unfinished requests remain explicitly unfinished rather than contributing observed completion latencies.

Policies see only arrived request identifiers, stable order, arrival time, prompt length, acknowledged progress, residency, last-output/prefill-completion anchor, and current time. Actual remaining output length, future arrivals, source identity, and output-derived metric deadlines are hidden. The engine either executes the exact proposed token spans or rejects the plan. The shared contract therefore fixes both the resources and the online information available to each adaptation.

3.3 Adaptation identities and structural consequences

Table 2 defines the four evaluated adaptations. Published systems are lineage references; historical project implementations and CM-0.1 adaptations are separate objects. The panel was targeted using the historical comparison questions, not sampled to represent all schedulers.

Two consequences are essential to interpreting the results. Let R be the resident count at a batch boundary and $K \in \{4, 8\}$ the reservation limit. The planner offers at most $K - R$ waiting requests a slot; a completing request releases its slot only after that batch's work. Each selected request appears once, contributing at most 512 prefill tokens or one decode token. Thus the bound is $512[R + (K - R)] \leq 4,096$, including both existing residents and new admissions, rather than a separate allowance for each. The 32,768-token budget cannot truncate this work.

Starting from the same state, chunked-FCFS and decode-first select the same residents and the same stable-order waiting prefix. They execute identical per-request spans, although the span lists can have different orders. The engine gives every span the same batch-end timestamp and releases completed reservations afterward. Consequently their next

Table 2. Project-maintained adaptations. Artifact IDs append `_c01` to `whole_fcfs`, `chunked_fcfs`, `decode_first`, and `slo_gated`. All use the shared contract above.

Adaptation	Historical lineage	Executed ordering rule and principal restriction
Whole-FCFS	vLLM-inspired whole-prompt implementation	Admit whole prompts in arrival order when feasible; otherwise decode residents. Admitting prefills defers all decodes that batch. No native paged allocator.
Chunked-FCFS	vLLM-inspired chunked implementation	Resident work before waiting work in stable order; at most 512 prefill tokens per request per batch.
Decode-first	Sarathi-inspired implementation	Resident decodes, then resident prefills, then waiting prefills; 512-token cap. No original engine timing.
SLO-gated	SLAI-inspired implementation	Urgent decodes first; then resident prefills, waiting prefills by prompt length, and residual decodes. Fixed 0.495 s gate; 512-token cap. No dynamic offset, heterogeneous classes, or full SLAI.

states, admissions, and timestamps agree inductively. The complete saved terminal records confirm this configuration-specific equivalence in all 1,440 matched conditions; it is not a theorem about the original serving systems.

The same work bound includes SLO-gated's residual decodes, even though they follow waiting prefills. A newly decode-ready resident's anchor is its prefill batch's end; every later decode resets it to the next batch end. There is no idle jump while such a resident exists. At each scheduling boundary its anchor age is therefore zero under the implemented clock updates, far below 0.495 s. The urgency gate never binds. Prompt-length ordering of waiting admissions remains active; resident prefills still progress, and later arrivals cannot preempt admitted requests. Whole-FCFS also retains a distinct rule: admitting whole prefills defers all decodes in that batch. Neither mechanism is removed by the work bound. Thus the model does more than give policies identical settings: it identifies which of their decision rules can change state transitions. The resulting finite-window completion distributions are the empirical object of Section 5.

3.4 Conformance evidence

The event referee reconstructs request states, token spans, timing, completion, and resource use from the inputs and execution journal without importing the engine or policy implementation. It checks that intended and executed work agree and recomputes the metrics. Every campaign cell passed this check; an independent analyzer also checked the complete ranking and reversal reports. The final evidence review additionally checked all result identities and terminal metrics and replayed 24 structurally selected new-cell journals. These checks establish conformance to CM-0.1, not GPU fidelity. Figure 1 summarizes the evidence chain.

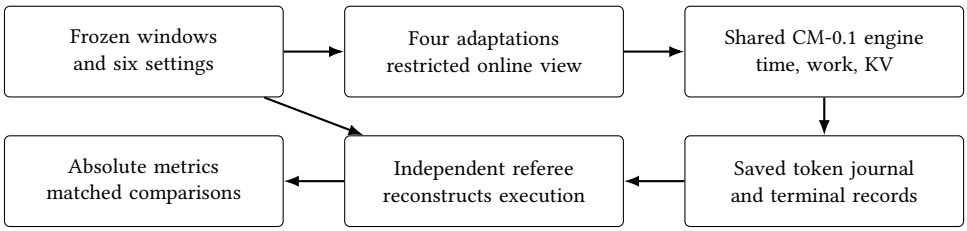


Fig. 1. Comparison and verification under one model. The independent analysis check recomputes aggregates, ranks, and reversals from the absolute metrics. No stage in this chain establishes physical-system accuracy.

Table 3. Consumed dataset provenance. Each row contributes 40 windows and 8,000 request records. Hashes are prefixes of SHA-256 values; full values are in the bundled input manifest. The artifact provenance audit matches these hashes to upstream release assets or a pinned LFS object and retains the corresponding license notices.

Source	File and representation	File hash prefix
Azure 2024	`AzureLLMInferenceTrace_conv_2024.csv`; source arrival and token counts	`a0cc9b969a9b`
Bailian/Qwen	`qwen_traceB_blksz_16.jsonl`; input/output lengths read directly; 16-token prefix-block hashes are separate fields	`68e3f98e2d60`
BurstGPT	`BurstGPT_without_fails_2.csv`; retained valid source rows and token counts	`56193aa9b2bb`

4 Corpus, Conditions, and Analysis

4.1 Trace and implementation provenance

The corpus comprises 40 windows of 200 requests from each of Azure 2024, Bailian/Qwen, and BurstGPT [5, 18, 24]. The Azure release identifies DynamoLLM as its accompanying publication [21]; the pinned Bailian/Qwen release requests attribution to the KVCache workload study [23]. These are three separately released sources, not three independent provider populations: BurstGPT and Azure have Azure infrastructure in their collection provenance. Table 3 identifies the consumed files and transformations. The supplement records full file and window fingerprints. The official Azure conversation release has the name `AzureLLMInferenceTrace_conv_1week.csv`; its published SHA-256 matches the consumed file despite the local rename. Azure and BurstGPT are licensed CC BY 4.0, and the pinned Bailian/Qwen data Apache 2.0. Input acquisition instructions retain attribution, license texts, and transformation notices. Public artifact distribution remains pending.

Each source's 40-window panel contains ten inherited pilot windows and 30 subsequent windows selected using fixed seeds and valid-row ranges. The expanded manifest was frozen before its corresponding campaign outcomes; earlier pilot outcomes already existed. Within the C01 manifest, row-overlap checks find no overlapping selected ranges. Exact overlap with some related projects' older BurstGPT selections remains unresolved. Nonoverlap within this panel does not establish temporal independence or representativeness. The frozen chronology labels allocate 20/10/10 windows per source to EARLY/MIDDLE/LATE. Four consecutive ten-window blocks provide an additional descriptive sensitivity; their elapsed time spans are unequal.

C01 reuses the historical input reconstruction, while its execution engine, policy views, and adaptation implementations have separate identities. C01-FULL retains C01 execution semantics at commit `9aed0206de66`; its full-campaign execution commit is `2d8342e699f3`. The complete matrix has $120 \times 6 \times 4 \times 2 = 5,760$ cells. Of these, 144 were reused from the pilot only after matching input, code, configuration, and journal identities; 5,616 were newly executed in the completed full campaign. Reuse is not an independent replication. No cell is omitted, and a deterministic cell is not rerun to manufacture a sampling distribution.

4.2 Inherited settings jointly transform arrivals and deadlines

The six labels LOW, PRE_KNEE, KNEE, POST_KNEE, OVERLOAD, and HIGH_PRESSURE come from historical calibration, not from a saturation analysis of CM-0.1. For a window-specific compression factor f, rebased arrival gaps and synthetic deadline slack are both divided by f. In idealized notation,

$$a'_i = (a_i - a_1)/f, \qquad d'_i = a'_i + (d_i - a_i)/f. \tag{1}$$

The implementation accumulates scaled nonnegative gaps in arrival order. Prompt/output lengths and priorities are unchanged. Factors range from 22.525 to 180.873 for Azure, 17.956 to 186.580 for Bailian, and 102.884 to 725.020 for BurstGPT. Equal labels therefore do not mean equal absolute arrival rates across windows or sources.

Deadlines are synthetic, derived in the inherited workload construction using output-length information; priorities are uniform. These attributes are not observed production service contracts. Deadlines are used only by the metric layer and are hidden from the scheduling adaptations. The setting axis is consequently *joint arrival/deadline stress*. A change in goodput across this axis cannot be attributed solely to arrival load.

4.3 Endpoints and aggregation

For arrival a_i, first output u_i, completion c_i, and output count o_i, latency is $c_i - a_i$ and time to first token (TTFT) is $u_i - a_i$. Time per output token (TPOT) is $(c_i - u_i)/(o_i - 1)$ for $o_i > 1$. The saved means and p95s use their defined completed-request populations; one-output requests do not have TPOT. The p95 uses the saved linear interpolation convention. Completion fraction is completed requests divided by all arrivals. Token throughput is completed output tokens divided by elapsed time from first arrival to final completion, or to the horizon if unfinished. Emitted-token throughput also counts any output of unfinished requests. Request throughput uses the same time denominator. These are finite-window completion rates, including the drain period; they do not estimate a maximum sustainable arrival rate or system capacity.

Arrival-normalized weighted goodput is

$$\mathrm{ANWG} = \frac{\sum_i w_i \, \mathbf{1}\{i \text{ completes and } c_i \le d'_i\}}{\sum_i w_i}. \tag{2}$$

Here $w_i = 1$, so ANWG is the fraction of arrivals meeting their synthetic deadline. It is dimensionless, unlike requests or tokens per second.

The eleven frozen ranking endpoints are ANWG, completion fraction, three throughput measures, and mean/p95 latency, TTFT, and TPOT. Larger is better for goodput, completion, and throughput; smaller is better for the six time endpoints. The complete endpoint set remains in the supplement, including redundant and numerically uninformative endpoints.

The window is the principal analysis unit. At a fixed source, setting, capacity, and policy, source means give each of the 40 windows equal weight. For the compact source–capacity table, we first average the six settings within each window and then average the 40 windows. A mean of window p95s is always labeled as such; it is not a pooled-request p95. Matched differences hold window, setting, and capacity fixed. We show all windows, not a selected favorable subset.

4.4 Ranking and practical-reversal definitions

Exact equality is primary: all maximizers or minimizers enter the top set, and tied policies receive average ranks. The frozen sensitivity admits values within $10^{-12}\max(1, |v_{\mathrm{best}}|)$ of the best; it does not replace exact rankings or the practical-margin rule.

For policies A, B and condition c, let $\Delta_c = q_m(v_{A,c} - v_{B,c})$, where $q_m = 1$ for a larger-is-better endpoint and $q_m = -1$ otherwise. The relative margin is

$$r_c = \frac{|v_{A,c} - v_{B,c}|}{|v_{\mathrm{loser},c}|}. \tag{3}$$

A practical reversal between c and c' requires $\Delta_c \Delta_{c'} < 0$ and both $r_c > 0.10$ and $r_{c'} > 0.10$. The strict boundary is evaluated exactly on the represented binary values. Zero losing values and missing values are unestimable. Nonzero ties remain eligible with margin zero. All denominators retain these distinctions.

Separate summaries compare within-source window pairs, Cartesian cross-source window pairs, settings within a window, capacities at a fixed window and setting, metrics at a fixed window, setting, and capacity, and source means at a fixed setting/capacity/metric. Cross-source windows are not paired by ordinal. For metric comparisons, each endpoint is oriented separately. These overlapping comparisons describe the finite corpus. We attach no population confidence intervals, p-values, or significance stars to C01.

5 Absolute Performance and Trade-offs (RQ1)

5.1 Completion and performance magnitudes

Every cell completes all 200 requests: 1,152,000 request executions across repeated conditions, not independent requests. There is no horizon censoring and no undefined cell-level latency, TTFT, or TPOT endpoint. TPOT populations range from 103 to 200 because single-output requests are excluded from that particular metric. Across cells, mean latency ranges from 0.008314 to 3.893923 s, p95 latency from 0.008560 to 7.300052 s, and token throughput from 332.489 to 7,766.685 tokens per simulated second.

Table 4 reports absolute source–capacity aggregates. Chunked-FCFS and decode-first share a row because their terminal records and all metrics are exactly equal; their separate identities remain in the artifact.

5.2 Mean latency gains do not imply a universal improvement

SLO-gated reduces aggregate mean latency against chunked-FCFS in all six source–capacity groups, by 11.3–34.0%. The largest aggregate reduction is Azure at four slots, from 2.8132 to 1.8559 s. At eight slots, BurstGPT changes from 0.1907 to 0.1692 s, the smallest percentage improvement. This is a substantial difference in the model's mean response time, but it comes with a throughput reduction in every group, ranging from 1.65% to 4.01%.

The tail comparison is less favorable. For BurstGPT, mean window-p95 latency rises from 1.0005 to 1.0377 s at four slots and from 0.3707 to 0.4410 s at eight slots: increases of 3.72% and 18.97%. Shorter average latency can therefore coexist with longer upper-tail latency under the same contract. At four-slot Bailian, ANWG is also 3.12% lower for SLO-gated despite its 24.05% mean-latency reduction. Throughput, deadline attainment, and response time reward different aspects of the completed schedule.

For a matched, fully completed 200-request condition, mean latency is $\sum_i (c_i - a_i)/200$, whereas token throughput is $\sum_i o_i / (\max_i c_i - \min_i a_i)$. Reducing the sum need not reduce the last completion or an upper quantile. Deadline attainment counts which completions cross synthetic deadlines; its Bailian loss is therefore compatible with the mean-latency gain. The source/capacity throughput means average window rates and are not inverses of mean durations. The following saved-record illustration connects these distinct objectives to the active admission ordering identified in Section 3.

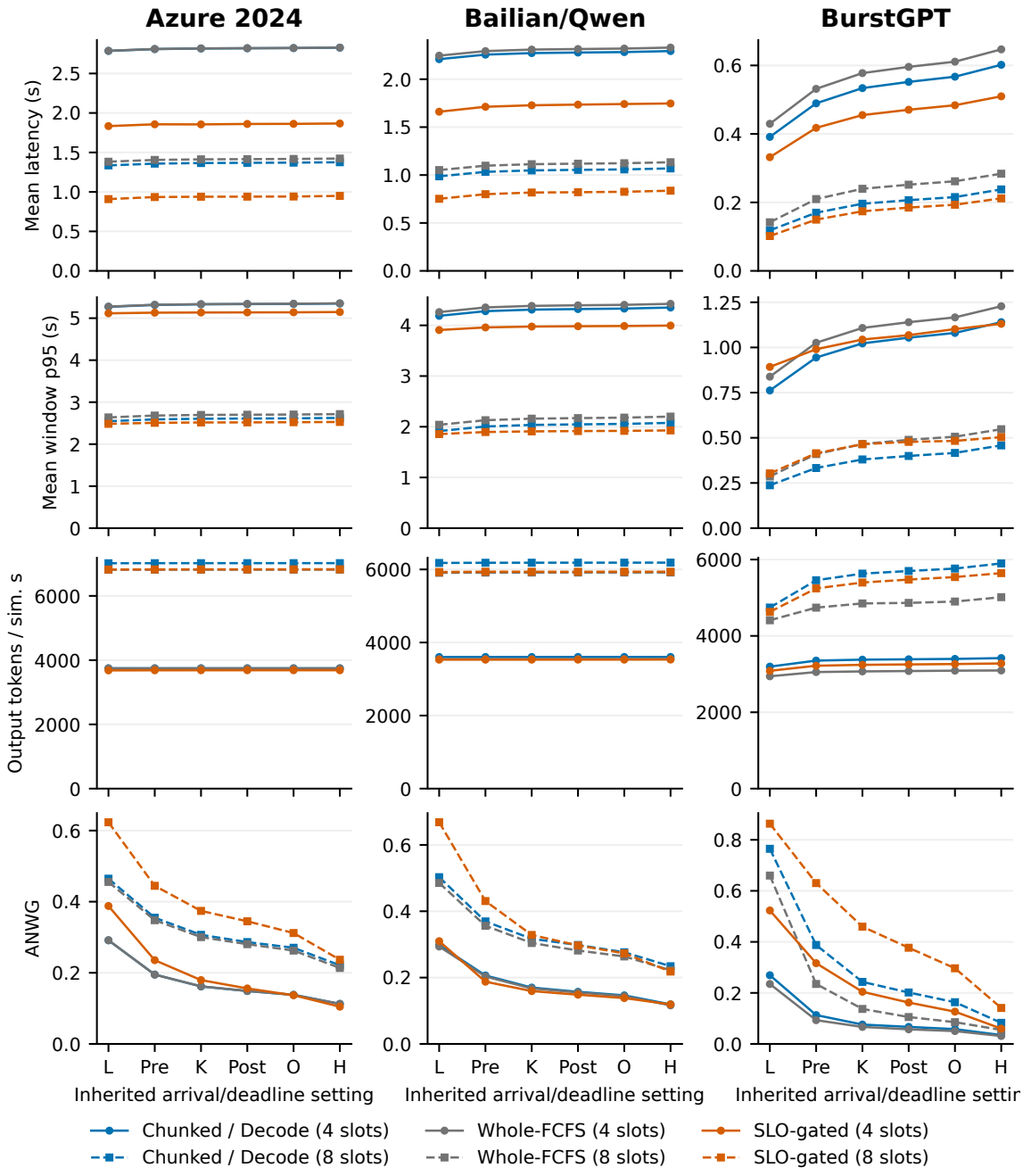


Fig. 2. Absolute performance across all six inherited settings, equally averaged over 40 windows per source. Solid circles: four slots; dashed squares: eight slots. L/Pre/K/Post/O/H abbreviate LOW through HIGH_PRESSURE in their declared order. Chunked and decode-first curves coincide exactly. The p95 panels average window-level p95s. Both arrivals and synthetic deadlines change along the horizontal axis. Lines connect descriptive condition means, with no inferential uncertainty bands.

Table 4. Absolute performance, equally averaging six settings within each window and 40 windows within each source. "Slots" are effective full-context residents. Mean and p95 columns are mean latency and *mean window-level p95 latency*; throughput is output tokens per simulated second.

Source	Slots	Adaptation	ANWG	Mean (s)	p95 (s)	Tokens/s
Azure 2024	4	Chunked / Decode	0.1746	2.8132	5.3232	3746.2
		Whole-FCFS	0.1746	2.8145	5.3275	3743.5
		SLO-gated	0.2001	1.8559	5.1342	3684.4
Azure 2024	8	Chunked / Decode	0.3174	1.3626	2.5986	7018.7
		Whole-FCFS	0.3101	1.4092	2.6892	6816.5
		SLO-gated	0.3892	0.9357	2.5131	6825.8
Bailian/Qwen	4	Chunked / Decode	0.1828	2.2655	4.2969	3603.2
		Whole-FCFS	0.1799	2.3020	4.3705	3548.3
		SLO-gated	0.1771	1.7205	3.9681	3531.4
Bailian/Qwen	8	Chunked / Decode	0.3330	1.0417	2.0227	6183.4
		Whole-FCFS	0.3189	1.1063	2.1463	5914.1
		SLO-gated	0.3693	0.8087	1.9037	5935.7
BurstGPT	4	Chunked / Decode	0.1030	0.5225	1.0005	3354.9
		Whole-FCFS	0.0889	0.5654	1.0843	3054.9
		SLO-gated	0.2321	0.4448	1.0377	3223.4
BurstGPT	8	Chunked / Decode	0.3072	0.1907	0.3707	5532.1
		Whole-FCFS	0.2129	0.2314	0.4505	4796.5
		SLO-gated	0.4613	0.1692	0.4410	5322.3

5.3 A saved-record explanation of the trade-off

Consider BurstGPT window `burstgpt_pilot_v2_w12`, setting `HIGH_PRESSURE`, capacity 262,144 (eight resident slots). We selected this illustration retrospectively: among BurstGPT conditions where SLO-gated has lower mean latency, lower throughput and higher p95 than chunked-FCFS, it is first in lexical window/setting/capacity order. This is an outcome-selected example, not a random or representative window. Its identity is frozen in `major-revision/example-manifest.json`.

Both adaptations complete the same 200 requests and emit 3,520 output tokens. Table 5 uses all requests: mean latency falls by 40.60%, but p95 rises by 33.17% and throughput falls by 14.63%. The throughput denominator runs from first arrival to final completion, including drain; it grows from 1.150 to 1.347 s. The p95 linearly interpolates ordered per-request latencies at zero-based position 189.05. It is a single-condition quantile, not the source mean of window p95s in Table 4.

Table 5. One outcome-selected BurstGPT condition, with identical completed inputs and output count. All times and rates are CM quantities. Duration includes the drain; percentages in the text are relative to chunked-FCFS.

Adaptation	Mean (ms)	p95 (ms)	Duration (s)	Tokens/s
Chunked-FCFS	61.681	110.741	1.150	3060.870
SLO-gated	36.641	147.480	1.347	2613.215

The saved journals show how admission competition remains active despite the nonbinding token budget. At 14 ms, request 5 has just completed, leaving one slot; requests 13 and 14 have both arrived. Chunked-FCFS admits request 13 (214 prompt tokens), whereas SLO-gated admits request 14 (201 tokens). The other request enters at 16 ms in each run. This is the first difference in admitted membership, not merely in the order of spans within a batch. Table 6 includes this

pair, the request with the largest completion delay (26), and the request with the largest improvement (196). Waiting here means arrival to the start of first prefill, not TTFT or total latency; completion timestamps are measured from the common origin.

Table 6. Selected request timelines from the same journals. C: chunked-FCFS; S: SLO-gated. Request IDs are zero-based saved input IDs. Output lengths are used retrospectively for explanation and were hidden from both policies.

ID	Prompt/output (tokens)	Arrival (ms)	Wait to prefill		Completion time	
			C (ms)	S (ms)	C (ms)	S (ms)
13	214/7	13.399	0.601	2.601	22.000	24.000
14	201/7	13.399	2.601	0.601	24.000	22.000
26	1976/1120	24.883	1.117	198.117	1150.000	1347.000
196	189/7	178.009	104.991	1.991	291.000	188.000

Request 26 has a 1,976-token prompt and 1,120 output tokens. Its first prefill starts at 26 ms under chunked-FCFS and 223 ms under SLO-gated; its first output occurs at 31 and 228 ms, respectively. It completes last in both runs, at 1.150 and 1.347 s. Thus its later service accounts exactly for the longer denominator in this condition, without changing total output. The admission rule observes its prompt length, not its output demand. Across all requests, 151 complete earlier under SLO-gated, 29 later and 20 at the same time; the sum of latency changes is −5.008 s. Mean latency therefore improves even while the last completion and upper quantile worsen. The p95 brackets involve other requests (191/195 under C and 139/68 under S), so the final request alone does not explain the p95 change. These timelines explain compatibility of the objectives in a real saved condition; the few displayed requests do not exhaustively decompose the aggregate change or estimate causal mechanism effects across the corpus.

The example manifest records exact input/result/journal identities and SHA-256 hashes. Independent event reconstruction checks all 400 terminal states and 2,497 events against the displayed tables; the complete records support reproduction of the illustration without selecting a new condition.

5.4 Capacity and inherited-setting effects

Increasing reservation capacity doubles permitted residency from four to eight. In every source–adaptation aggregate shown, mean and mean-p95 latency fall and throughput rises. For chunked-FCFS, Azure mean latency falls from 2.8132 to 1.3626 s while throughput rises from 3,746.2 to 7,018.7 tokens/s; BurstGPT changes from 0.5225 to 0.1907 s and from 3,354.9 to 5,532.1 tokens/s. The capacity axis is predominantly a concurrency constraint in this model. The largest observed used KV is 47,872 tokens in the four-slot profile and 62,880 in the eight-slot profile, both below even the smaller nominal capacity. These results do not measure native allocator pressure, eviction, or the benefit of paging at an exhausted used-token limit.

Figure 2 separates settings within each source–capacity. The inherited setting labels can produce large changes in goodput with small changes in latency or throughput. For Azure chunked-FCFS at four slots, LOW to HIGH_PRESSURE changes mean latency from 2.7858 to 2.8269 s and throughput from 3,745.9 to 3,746.3 tokens/s, while ANWG falls from 0.291375 to 0.112625. Since the transformation also contracts deadline slack, this is not evidence for an isolated arrival-load effect on goodput.

5.5 Matched-window variation

Figure 3 displays all 40 window-level matched differences per source and capacity, averaging the six paired setting differences within each window. It complements the source means by showing where a performance difference changes sign or is concentrated. The vertical zero line represents equal performance; it is neither a confidence bound nor the 10% practical-reversal threshold. For latency, a negative difference favors SLO-gated; for throughput, it favors chunked-FCFS.

At the unaveraged condition level, SLO-gated uniquely minimizes mean latency in all 480 Azure conditions, 468 of 480 Bailian conditions, and only 198 of 480 BurstGPT conditions. Its favorable source aggregates therefore do not mean that it wins every window/setting comparison. The tail and throughput distributions make the same distinction visible without reducing each comparison to a winner label.

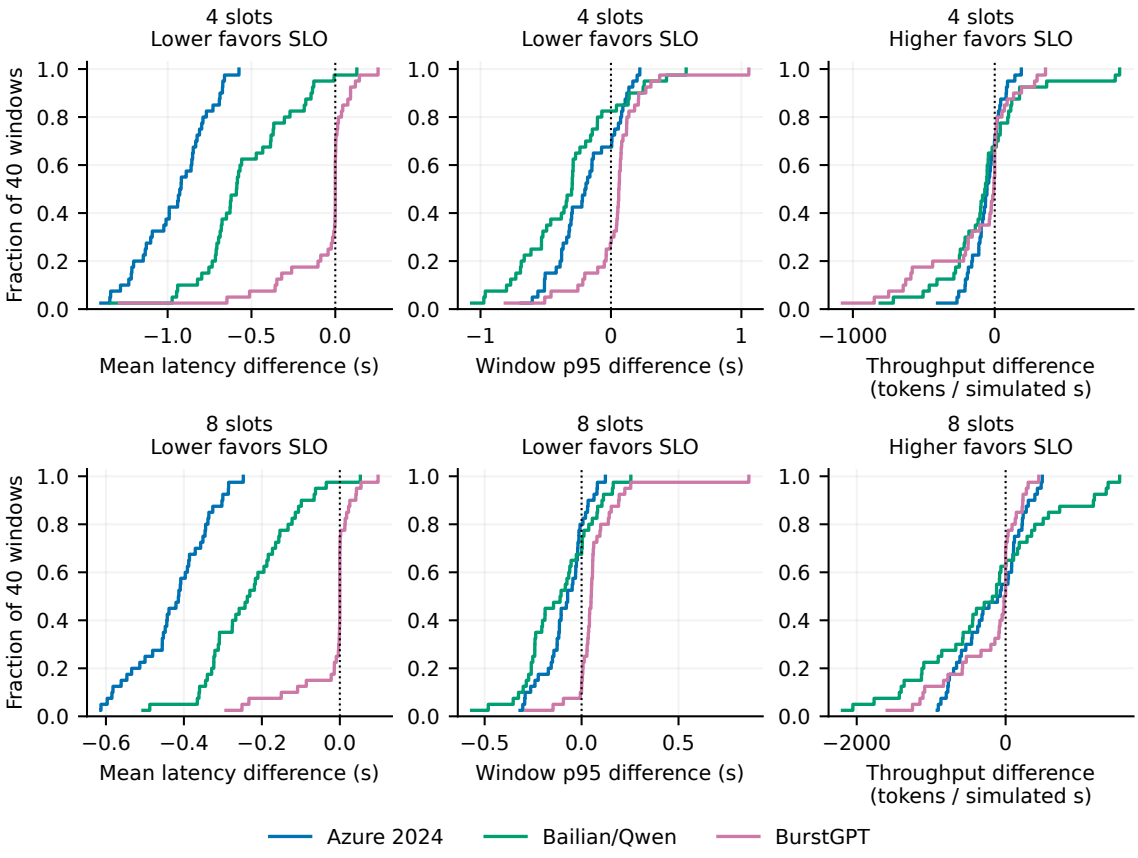


Fig. 3. Empirical distributions of SLO-gated minus chunked-FCFS differences. Each curve has 40 equally weighted windows; each point averages six matched setting differences within one window. The axes show absolute differences, not percentages. Positive latency differences and negative throughput differences are unfavorable to SLO-gated. All windows are retained.

Answer to RQ1. The surviving behaviors exchange average response time against throughput, tail latency, and deadline attainment. The capacity contrast is principally a residency constraint; the setting contrast jointly stresses arrivals and deadlines. These are trade-offs within the declared model.

Table 7. Strict-10% practical reversals. Nonzero ties remain in eligible denominators; a zero losing value is unestimable. All counts include six policy pairs and the declared endpoints. Reused windows and overlapping pairs make these dependent descriptive counts, not independent trials.

Comparison axis	Reversals	Eligible	Total	Unestimable
Within-source windows	34,394	1,847,700	1,853,280	5,580
Cross-source windows	109,913	3,788,640	3,801,600	12,960
Inherited settings	920	236,853	237,600	747
KV capacities	350	47,418	47,520	102
Metric pairs	11,795	473,580	475,200	1,620
Source means	0	2,376	2,376	0

6 Rankings, Aggregation, and Metric Choice (RQ2)

6.1 Stable practical source comparisons and heterogeneous windows

Table 7 reports the strict-10% reversal rule with eligible and total denominators. Across all source-mean comparisons the count is **zero out of 2,376**. Each source mean uses all 40 windows at a fixed setting, capacity, and metric. The denominator is 3 source pairs ×6 settings ×2 capacities ×11 endpoints ×6 policy pairs. For this equal-window estimand, changing source does not overturn a policy-pair preference by more than the declared margin in both directions. The 2,376 comparisons include the equivalent chunked/decode-first pair and redundant throughput endpoints; they do not constitute 2,376 independent confirmations or establish population equivalence.

This null result is compatible with smaller sign changes and different tie sets. Among 396 comparisons of source pairs at fixed settings, capacities, and metrics, exact top sets are equal in 307 and intersect in 323; complete average-rank vectors are equal in 247. Absence of a strict practical reversal is weaker than equality of rankings or top sets.

Window-level variation remains visible. Within-source window comparisons yield 34,394 reversals among 1,847,700 eligible comparisons; Cartesian cross-source window comparisons yield 109,913 among 3,788,640. For ANWG alone, the corresponding counts are 8,472/162,900 and 35,352/332,640. The larger window-pair denominators arise from comparing every unordered pair of 40 windows within a source (780) or every Cartesian pair across sources (1,600). They are not larger independent sample sizes. These summaries cannot establish that one axis causally contributes more instability than another.

Within the same window, settings produce 920/236,853 eligible reversals, capacity changes 350/47,418, and metric choice 11,795/473,580. The last count compares policy-pair directions across endpoints; it demonstrates that a preference can depend on the objective even with the workload and resources fixed. It is distinct from comparing top sets, which asks whether at least one best policy is shared rather than whether a particular pair reverses.

6.2 Ties and redundant endpoints

Table 8 retains exact ties instead of breaking them by policy identifier. SLO-gated is uniquely best for mean latency in 1,146 conditions and for throughput in 482. Completion has a four-way tie everywhere.

All requests finish, making emitted-token and completed-token throughput identical. Request throughput differs only by a workload-specific multiplier shared by all policies at a fixed condition. The three throughput rankings therefore coincide. Chunked-FCFS, decode-first, and SLO-gated also produce essentially one output per millisecond after decoding begins: the largest chunked-versus-SLO mean-TPOT difference is 1.80×10^{-16} s. Timestamp subtraction

Table 8. Ranking summaries, each out of 1,440 window/setting/capacity conditions. "Near-tie change" counts conditions whose top set changes under the predeclared $10^{-12}\max(1, |v_{\text{best}}|)$ tolerance. No ranking is missing. "SLO unique top" always uses exact equality.

Endpoint	Exact top tie	SLO unique top	Near-tie change
ANWG	425	1,004	0
Mean latency	289	1,146	6
p95 latency	586	834	0
Token throughput	906	482	0
Completion	1,440	0	0
Mean TPOT	979	461	1,129
p95 TPOT	1,348	92	104

Table 9. Post hoc reduced view: three behavior pairs and eight endpoints. The original strict-10% rule is unchanged. These dependent descriptive counts supplement, rather than replace, Table 7.

Axis	Reversals	Eligible	Total
Source means	0	864	864
Within-source windows	18,732	670,637	673,920
Cross-source windows	61,880	1,374,720	1,382,400
Settings	629	85,962	86,400
Capacity	240	17,219	17,280
Metric	4,492	120,288	120,960

nevertheless changes exact top sets; the predeclared near-tie tolerance changes mean-TPOT top sets in 1,129 conditions and p95-TPOT top sets in 104. These tiny distinctions have no practical performance meaning.

Across all 79,200 metric-pair ranking comparisons, exact top sets are equal in 27,915 and intersect in 57,602. These counts include redundant throughput endpoints, completion ties, and numerical TPOT distinctions. They summarize reporting consequences; they are not eleven independent tests of a scheduler's robustness.

6.3 Post hoc view of distinct behaviors and metric families

To test whether duplication drives the qualitative ranking conclusions, we project the saved outputs onto whole-FCFS, chunked-FCFS, and SLO-gated. We remove the universally tied completion endpoint and retain token throughput as the representative of the three identical throughput rankings. The resulting eight endpoints cover five families: latency, TTFT, TPOT, throughput, and deadline attainment. Both mean and p95 remain for the three time families because they describe different distributional features. TPOT remains informative for whole-FCFS: its mean and p95 exceed the best value by the same strict-10% practical margin in 341 and 1,027 conditions, respectively. Removing TPOT altogether would discard that contrast.

This view contains 11,520 rankings, 34,560 matched pairwise records, and 8,208 reversal groups. Independent exact-boundary calculations from absolute metrics reproduce every retained archived group. Table 9 still has no source-mean practical reversals (0/864), but has window and metric reversals. The qualitative contrast therefore survives removal of these demonstrated redundancies. Rates change with the denominator; the remaining endpoints and overlapping comparisons are not independent replications. The original four-label, eleven-endpoint outputs remain intact. Chronological sensitivities appear in Appendix C.

Table 10. Compact calibration diagnostic for the actual historical-source screened-BH comparator. Each row has 8,000 studies; ordinary fixed-contrast interval coverage is available in all of them. Bounds are the expanded 17-method simultaneous Monte Carlo bounds, not confidence intervals for C01 performance. These three null scenarios isolate IID, independent-block, and AR dependence; all 18 scenarios and methods remain in the supplement.

Scenario	FDR	MC bounds	Coverage	MC bounds
IID equal null	0.09025	[0.06878, 0.11172]	0.94187	[0.92357, 0.96018]
Independent-block null	0.91800	[0.89726, 0.93874]	0.64762	[0.61465, 0.68060]
AR(0.7) null	0.98037	[0.96780, 0.99295]	0.56487	[0.53081, 0.59894]

Answer to RQ2. Source-mean practical stability does not imply uniform preferences across windows or objectives. This remains true after collapsing duplicate labels and endpoints; magnitudes and top sets distinguish meaningful preferences from ties and numerical noise.

7 Historical Interpretation and Calibration (RQ3)

7.1 What the historical comparison establishes

The historical analysis reported 36 of 990 policy-pair/source-pair/setting records meeting its practical-margin and nominal-interval rule. Numerical reproduction confirms that archived-rule count. It does not establish population reversals: the historical policy–model combinations differ, selected trace windows have no known population sampling law, and the inference procedure's calibration was unestablished. The C01 panel and execution model change the comparison object, so its zero source-mean practical reversals are not a like-for-like disappearance of those 36 records. Historical and common-model outcomes are kept in separate evidence strata rather than combined into a leaderboard.

7.2 A focused known-truth diagnostic

STAT01 evaluates historical inference in synthetic studies whose population means, covariance, and missingness are specified. Each study has three sources, eleven policies, and normally 40 windows per source; one bounded scenario uses 400. The 165 source-pair/policy-pair hypotheses are dependent within a study. Independently generated studies, not their individual hypotheses, are the Monte Carlo units. The completed design contains 134,000 studies across 18 scenarios, including IID, block-dependent, autoregressive, nonnormal, bounded, and missing-data cases.

The primary truth is a population reversal exceeding 10% relative to the losing mean in both sources. Sign reversals are a separate endpoint. The historical procedure first screens noisy point estimates at 10%, then uses bootstrap intervals and zero-effect p-values on the selected contrasts. Testing a zero-effect null after such screening is not automatically a test of the practical-effect null. BH or BY alone cannot supply the missing calibration [7, 8].

Three implementation identities are retained. The eight frozen original procedures include a port of the historical screened-BH procedure. A post-diagnosis comparator reproduces the actual historical source's floating-point evaluation order. Eight additional comparisons implement explicit mathematical threshold semantics. Thus the expanded family has 17 methods, with nine additions identified as post-diagnosis. Arithmetic correction is not a new test and does not change the inferential assumptions. Appendix A gives the definitions and the saved boundary case.

Table 10 shows that the actual historical-source procedure has practical FDR 0.09025 in the IID equal-null scenario, with an expanded Monte Carlo lower bound above 0.05. Independent-block and AR nulls show much larger false-discovery rates and poor ordinary interval coverage. These null examples already demonstrate a calibration concern without requiring interpretation of power or near-boundary alternatives.

Appendix A retains all classifications, stopping counts, original-eight and expanded-17 uncertainty families, abstentions, precision gaps, and low-power outcomes. These qualify the calibration findings.

Answer to RQ3. Reproducing historical arithmetic establishes what its rule reported; execution conformance establishes that C01 follows its common model. Neither establishes valid statistical inference. STAT01 demonstrates calibration failures for historical procedures in explicit known-truth scenarios, while retaining uncertainty and assumption-specific limits. C01's finite-corpus descriptions do not depend on population significance tests.

8 Discussion and Threats to Validity

What is learned from controlling execution semantics? The useful comparison unit is an executable behavior, not a policy name. Here that distinction has a concrete consequence: nominal priority changes can leave every request outcome unchanged while admission competition still changes the completion distribution. Section 3 establishes the structural conditions; Sections 5 and 6 quantify the remaining trade-offs. The post hoc reduction shows that the source-mean/window contrast survives removal of duplicated comparisons. These results go beyond a warning about unfair settings by identifying exactly which distinctions disappear in this implemented contract and what remains measurable.

The competing objectives themselves are established scheduling ideas. The BurstGPT example contributes a traceable instance: many earlier completions lower the sum of latencies even as a delayed final completion extends the drain. It illustrates compatibility of the measured objectives, not optimality of prompt-length ordering or a corpus-wide causal effect. Similarly, source means answer an equal-window average question; window reversals describe variation concealed by that average. Neither summary supersedes the other, and exact top sets preserve information lost by arbitrary tie-breaking.

Model adequacy and transfer. Fixed batch timing removes composition-dependent cost, nonbinding token work exposes the role of admission, and conservative reservations provide a known residency constraint without output-length prediction. Together these choices support a precise analysis of state-transition distinctions. They omit kernel, bandwidth, communication, native allocation, and changing batch-efficiency costs. The proof's conditions delimit transfer: a binding work budget or composition-dependent timing can invalidate its argument, but this study does not measure the resulting behavior. Independent conformance checks establish implementation correctness for the abstraction, not physical prediction accuracy [20]. The findings provide a comparison diagnostic to apply before interpreting a ranking, rather than a deployment recommendation.

Corpus and inference limits. The selected windows are finite and temporally related, and the Azure-linked sources are not independent provider populations. Nonoverlap inside the C01 panel does not settle overlap with older cross-project BurstGPT selections. Uniform priorities and synthetic output-derived deadlines restrict goodput interpretation; the inherited transformation changes deadlines and arrivals together. C01 consequently reports descriptive finite-corpus effects. The reduced analysis is post hoc and removes only demonstrated redundancies; its remaining metric families and overlapping comparisons are still dependent.

STAT01's known truth applies only to its generators. Post-diagnosis methods, finite study counts, conditional interval availability, and conservative power remain explicit in Appendix A. Reproducible arithmetic and valid population inference are distinct requirements. The extensive checks reduce implementation and reporting risk without adding scientific novelty or resolving model adequacy by themselves.

9 Conclusion

This common-contract comparison links a structural reduction of scheduler labels to a smaller, interpretable performance problem. Two adaptations have identical terminal outcomes despite potentially different journal ordering; an inactive urgency gate leaves admission ordering as an active distinction. The surviving behaviors exhibit measured mean/tail/throughput trade-offs. Source-mean practical stability coexists with window and metric reversals even after duplicated comparisons are removed. These bounded findings make explicit what the evaluator permits a scheduling choice to change. The supporting historical diagnostic separately shows why exact arithmetic reproduction is insufficient evidence of statistical calibration.

Code and Data Availability

The manuscript's reproduction materials include source, figure and table scripts, frozen analysis summaries, input fingerprints, and supplementary CSVs containing all 5,760 C01 cells and 7,200 STAT01 endpoint summaries. A separately assembled local artifact contains the pinned execution code and complete saved campaign records. Regenerating displays from those records is distinct from acquiring upstream inputs and replicating the experiments. Versioned public distribution of the complete research artifact is pending; no archival DOI is asserted. Original third-party raw workload traces are excluded from the planned release and must be acquired from the attributed upstream sources. The intended license scopes are MIT for original code and CC BY 4.0 for eligible original research data and results, with upstream terms retained.

Declarations

No external research funding specifically supported this work. The author declares no competing interests and acknowledges NJIT's Wulver HPC computing resources.

AI assistance. OpenAI Codex was used to assist with software development, experimental analysis and validation, preparation of figures and tables, and manuscript revisions. OpenAI ChatGPT was used to assist with scientific brainstorming, manuscript organization, and suggestions for presenting and explaining the research. The author reviewed the resulting work and accepts full responsibility for the manuscript's content, scientific accuracy, and conclusions.

References

[1] Amey Agrawal, Anmol Agarwal, Nitin Kedia, Jayashree Mohan, Souvik Kundu, Nipun Kwatra, Ramachandran Ramjee, and Alexey Tumanov. 2024. Etalon: Holistic Performance Evaluation Framework for LLM Inference Systems. arXiv:2407.07000v2. https://arxiv.org/abs/2407.07000v2

[2] Amey Agrawal, Nitin Kedia, Anmol Agarwal, Jayashree Mohan, Nipun Kwatra, Souvik Kundu, Ramachandran Ramjee, and Alexey Tumanov. 2025. On Evaluating Performance of LLM Inference Serving Systems. arXiv:2507.09019v1. doi:10.48550/arXiv.2507.09019

[3] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. VIDUR: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of Machine Learning and Systems (MLSys '24)*, Vol. 6. MLSys, 351–366. https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html

[4] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, Santa Clara, CA, 117–134. https://www.usenix.org/conference/osdi24/presentation/agrawal

[5] Alibaba Cloud. 2026. *qwen-bailian-usagetraces-anon.* https://github.com/alibaba-edu/qwen-bailian-usagetraces-anon/tree/5f7439c51ec248a0c585f7d90a41a6f57773b912 Anonymized Bailian/Qwen production LLM-serving usage-trace release; repository `alibaba-edu/qwen-bailian-usagetraces-anon`, pinned commit `5f7439c51ec248a0c585f7d90a41a6f57773b912`; Apache License 2.0; consumed trace identity verified against the pinned Git LFS object.

[6] Agrim Bari, Parikshit Hegde, and Gustavo de Veciana. 2025. Optimal Scheduling Algorithms for LLM Inference: Theory and Practice. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 3, Article 59 (2025), 43 pages. doi:10.1145/3771574

[7] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B (Methodological)* 57, 1 (1995), 289–300. doi:10.1111/j.2517-6161.1995.tb02031.x

[8] Yoav Benjamini and Daniel Yekutieli. 2001. The Control of the False Discovery Rate in Multiple Testing under Dependency. *The Annals of Statistics* 29, 4 (2001), 1165–1188. doi:10.1214/aos/1013699998

[9] Jaehong Cho, Hyunmin Choi, Guseul Heo, and Jongse Park. 2026. LLMServingSim 2.0: A Unified Simulator for Heterogeneous and Disaggregated LLM Serving Infrastructure. In *2026 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, Seoul, South Korea, 1–14. doi:10.1109/ISPASS69572.2026.00012

[10] Jaehong Cho, Minsu Kim, Hyunmin Choi, Guseul Heo, and Jongse Park. 2024. LLMServingSim: A HW/SW Co-Simulation Infrastructure for LLM Inference Serving at Scale. In *2024 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, Vancouver, BC, Canada, 15–29. doi:10.1109/IISWC63097.2024.00012

[11] J. G. Dai, Tianze Deng, Yueying Li, and Tianyi Peng. 2026. Throughput-Optimal Scheduling Algorithms for LLM Inference and AI Agents. arXiv:2504.07347v3. https://arxiv.org/abs/2504.07347v3 Version 3, May 18, 2026.

[12] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Vancouver, BC, Canada, 59006–59029. doi:10.52202/079017-1882

[13] Wassily Hoeffding. 1963. Probability Inequalities for Sums of Bounded Random Variables. *J. Amer. Statist. Assoc.* 58, 301 (1963), 13–30. doi:10.1080/01621459.1963.10500830

[14] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. https://www.jstor.org/stable/4615733

[15] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Xuefei Ning, Guohao Dai, Shengen Yan, Yun Liang, and Yu Wang. 2025. SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling. In *Proceedings of Machine Learning and Systems*, Vol. 7. MLSys, Santa Clara, CA, 13 pages. https://proceedings.mlsys.org/paper_files/paper/2025/hash/bc82dbfbfa43232be85b8d9838f49c3e-Abstract-Conference.html

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, Koblenz, Germany, 611–626. doi:10.1145/3600006.3613165

[17] Andreas Maurer and Massimiliano Pontil. 2009. Empirical Bernstein Bounds and Sample Variance Penalization. In *Proceedings of the 22nd Annual Conference on Learning Theory*. 9 pages. https://www.cs.mcgill.ca/~colt2009/papers/012.pdf

[18] Microsoft Azure. 2024. *AzureLLMInferenceDataset2024*. https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2024.md

[19] Konstantinos Papaioannou and Thaleia Dimitra Doudali. 2024. The Importance of Workload Choice in Evaluating LLM Inference Systems. In *Proceedings of the 4th Workshop on Machine Learning and Systems (EuroMLSys '24)*. ACM, Athens, Greece, 39–46. doi:10.1145/3642970.3655823

[20] Robert G. Sargent. 2013. Verification and validation of simulation models. *Journal of Simulation* 7, 1 (2013), 12–24. doi:10.1057/jos.2012.20

[21] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, Las Vegas, NV, 1348–1362. doi:10.1109/HPCA61900.2025.00102

[22] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, Santa Clara, CA, 173–191. https://www.usenix.org/conference/osdi24/presentation/sun-biao

[23] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 465–482. https://www.usenix.org/conference/atc25/presentation/wang-jiahao

[24] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25), Volume 2*. ACM, Toronto, ON, Canada, 5831–5841. doi:10.1145/3711896.3737413

[25] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. 2026. ServeGen: Workload Characterization and Generation of Large Language Model Serving in Production. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, Renton, WA, 1845–1859. https://www.usenix.org/conference/nsdi26/presentation/xiang-servegen

[26] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association, Carlsbad, CA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[27] Netanel Zakay and Dror G. Feitelson. 2014. Workload resampling for performance evaluation of parallel job schedulers. *Concurrency and Computation: Practice and Experience* 26, 12 (2014), 2079–2105. doi:10.1002/cpe.3240

[28] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, Santa Clara, CA, 193–210. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

A Statistical Diagnostic: Definitions and Complete Scenario Summary

A.1 Study construction and truth

A study draws a latent array $Y \in \mathbb{R}^{3\times n\times 11}$, with positive population means μ_{sj}. Sources are independent within this synthetic design. Dependence across policies is $C(\rho) = (1-\rho)I + \rho\mathbf{1}\mathbf{1}^{\mathsf{T}}$, usually with $\rho = 0.5$. Temporal covariance is identity, independent five-window blocks with within-block correlation 0.7, or AR(1) with correlation $0.7^{|i-j|}$. Source-specific innovations have mean zero and variance one; a Cholesky transformation imposes the specified covariance, with default scale $\sigma = 0.15$. The exact-ties scenario uses $\rho = 1$.

Each family comprises $\binom{3}{2}\binom{11}{2} = 165$ hypotheses. For a source pair (s,t) and policy pair (a,b), sign-reversal truth is $(\mu_{sa} - \mu_{sb})(\mu_{ta} - \mu_{tb}) < 0$. Practical-reversal truth is

$$\begin{aligned}&(\mu_{sa} > 1.1\mu_{sb} \ \wedge \ \mu_{tb} > 1.1\mu_{ta})\\ &\quad \vee \ (\mu_{sb} > 1.1\mu_{sa} \ \wedge \ \mu_{ta} > 1.1\mu_{tb}).\end{aligned} \tag{4}$$

Equality at 10% is null. Truth is enumerated from decimal/rational mean specifications, independently of estimated means and test decisions.

Equal-null means are 0.5. Stable and near-tie means are $0.5 + 0.01j$ and $0.5 + 0.0002j$, respectively. A reversal of size δ assigns source 0 policies 0/1 means $0.5 \pm \delta/2$, swaps them in source 1, and sets all other means to 0.5. Small, medium, and large IID reversal scenarios use $\delta = 0.03, 0.10, 0.30$. The boundary scenario uses 0.55/0.50 and the reverse. Block, AR, unequal-variance, mixed-distribution, and MCAR reversal scenarios use $\delta = 0.15$. Unequal scales are $(0.08, 0.25, 0.15)$; mixed innovations are standardized normal, Student-t_5, and lognormal. MCAR retains each policy/window observation independently with probability 0.8. Selective missingness removes source 0/policy 0 and source 1/policy 1 observations below their respective latent means; truth remains the unconditional mean. Observed values cannot identify that target merely by ignoring missingness.

The bounded scenarios use IID standardized uniform innovations, $\sigma = 0.08$, and $\delta = 0.30$, giving scores in $[0,1]$ analytically, without clipping. All scenarios have 40 windows per source except the last, which has 400. Raw arrays and masks are retained in the full local artifact candidate. The 112,275 reused and 21,725 subsequently generated studies sum to the completed 134,000-study design; no study was removed because of a numerical disagreement between implementations.

A.2 Frozen methods and post-diagnosis additions

All methods use nominal level $\alpha = q = 0.05$. The historical screened procedure uses 2,000 bootstrap draws of matched observed per-window policy differences, percentile intervals, and $p = \min\{1, 2\min[\Pr_{\text{boot}}(\bar{D} \le 0), \Pr_{\text{boot}}(\bar{D} \ge 0)]\}$. It combines source-side p-values by their maximum, applies BH to the point-screened family, and requires both intervals to exclude zero. The full-family variant assigns untested hypotheses p-value one and applies BH over all 165. The historical plus-one BY variant transforms the component bootstrap p-value as $\min\{1, (2000p + 2)/2001\}$ before source combination and applies BY over 165. These modifications do not make selection or the original p-values valid by definition.

The analytic practical-effect methods use paired contrasts $Z = Y_a - 1.1Y_b$ and their reversed orientation. Each orientation combines the two one-sided source tests by a maximum (an intersection–union test). The union over orientations uses $\min\{1, 2\min(p_1, p_2)\}$ before Holm correction over 165 hypotheses [14]. IID t-Holm uses paired Student tests; cluster t-Holm uses independent five-window cluster averages, retaining only completely observed clusters. Fewer than two complete clusters causes abstention. Their finite-sample Gaussian justifications apply to their respective independent sampling units with complete or MCAR observations, not arbitrary temporal dependence. The covariance-oracle normal variant uses the known Gaussian covariance of each observed matched sample mean. It does not access missing latent outcomes and is not deployable without that covariance information.

Grouped-bootstrap BY resamples nonoverlapping five-window groups jointly across policies, preserving masks and pairing. Its centered one-sided component p-value is $(1 + \#\{\bar{Z}^* - \bar{Z} \geq \bar{Z}\})/2001$. A zero-observation resample causes component abstention; it is not silently excluded. The same orientation construction is followed by full-family BY. No finite-sample guarantee was prespecified for this procedure. Bounded Hoeffding–Holm applies only to the bounded IID scenarios. Since $Z \in [-1.1, 1]$, its component p-value is $\exp[-2m\max(\bar{Z}, 0)^2/2.1^2]$ [13], followed by the same orientation and Holm construction. It abstains outside that domain.

The eight original identities are retained unchanged. The ninth method, `legacy_source_screened_BH`, reproduces the actual frozen historical source's arithmetic. Eight `exact_` counterparts of the original methods use explicit rational critical thresholds and exact bootstrap-count grids. Analytic p-values are treated as the exact dyadic values represented by their finite floats, not as infinitely accurate analytic probabilities. Rational harmonic constants define BY thresholds. These additions change numerical semantics, not the sampling assumptions, screening target, or statistical validity of a component test.

A.3 Coverage, availability, power, and Monte Carlo accounting

For each independent study, let R be discoveries and V false discoveries. False-discovery proportion is $V/\max(R, 1)$; its expectation is FDR. Familywise error is $\mathbf{1}\{V > 0\}$, and power is true discoveries relative to the number of alternatives when alternatives exist. Both practical and sign truth are retained. Ordinary interval coverage uses a fixed source 0/policy 0-minus-policy 1 contrast. For historical methods, that interval is generated in a separate unconditional diagnostic stream. Coverage of actually selected intervals is a different, selection-conditional endpoint; it is not replaced by ordinary coverage.

Interval availability is reported separately. In the MCAR and selectively missing scenarios, cluster-t ordinary coverage is based on 1,676 and 202 of 8,000 studies, respectively. Historical selected-interval coverage in the IID equal null is 0.123834 over 958 studies emitting selected intervals, whereas ordinary fixed-contrast coverage is 0.941875 over 8,000 studies. These are study-level averages, not independent interval observations. The complete endpoint CSV also includes null rejection fractions, maximizer-set coverage and size, and deterministic-ID correctness; exact empirical maximizing sets are not confidence sets.

The five possible analysis looks are 500, 1,000, 2,000, 4,000, and 8,000 studies per scenario. For a bounded study-level endpoint with sample variance s^2 and available count N, the two-sided empirical Bernstein half-width is

$$b = \sqrt{\frac{2s^2 \log(4/\delta)}{N}} + \frac{7\log(4/\delta)}{3(N-1)}. \tag{5}$$

The bounds clip $\bar{X} \pm b$ to $[0, 1]$; empty endpoints are unestimated with the full range. The construction uses the empirical Bernstein bound and a finite union over both tails and all reported looks [17]. Conditional coverage uses the count of

IID studies emitting an interval, with that availability explicit. The original allocation is $\delta = 0.05/(18 \cdot 8 \cdot 16 \cdot 5)$; the expanded report uses $0.05/(18 \cdot 17 \cdot 16 \cdot 5)$. Dependence among methods and endpoints does not invalidate the union bound. The wider accounting does not erase post-diagnosis method selection.

Stopping uses only the original eight methods. Required half-widths are 0.025 for practical FDR, familywise error, and null rejection fraction; 0.03 for ordinary interval coverage; and 0.08 for defined power. Selected-interval coverage is not a stopping target. The calibration screen flags concern if the practical-FDR lower bound exceeds 0.05 or ordinary-coverage upper bound is below 0.95. Empirical compatibility with tolerance requires FDR upper bound at most 0.06 and coverage lower bound at least 0.94 in relevant assumption-valid scenarios. All other estimable cases are inconclusive. The screen permits conservative overcoverage and is not a proof of exact nominal calibration.

Table 11. Complete original-eight scenario screen and stopping counts. C: calibration concern; I: inconclusive; E: empirical compatibility with tolerance; A: outside-domain abstention. P: original precision objective met; M: maximum-count stop with gaps. H-S/H-F/H-BY: historical screened BH/full BH/plus-one BY; t/c-t/O/G/Hf: IID t/cluster t/oracle normal/grouped bootstrap/bounded Hoeffding. These use the *original-eight* uncertainty family. Full numerical results for all 17 methods and both families are in `stat01-all-endpoints.csv`.

ID	Scenario	Studies	Stop	H-S	H-F	H-BY	t	c-t	O	G	Hf
0	iid_equal_null	8,000	P	C	I	I	I	I	I	C	A
1	iid_stable_null	8,000	P	I	I	I	I	I	I	C	A
2	iid_near_tie_null	8,000	P	C	I	I	I	I	I	C	A
3	exact_ties_null	2,000	P	E	E	E	E	E	E	E	A
4	practical_boundary	8,000	M	C	I	I	I	I	I	C	A
5	small_sign_reversal	8,000	M	C	I	I	I	I	I	C	A
6	medium_reversal	8,000	M	C	C	I	I	I	I	C	A
7	large_reversal	8,000	M	I	I	I	I	I	I	C	A
8	block_null	8,000	M	C	C	C	C	I	I	C	A
9	block_reversal	8,000	M	C	C	C	C	I	I	C	A
10	ar_null	8,000	M	C	C	C	C	C	I	C	A
11	ar_reversal	8,000	M	C	C	C	C	C	I	C	A
12	unequal_variances	8,000	M	I	I	I	I	I	I	C	A
13	mixed_distributions	8,000	M	I	I	I	I	I	I	C	A
14	mcar_reversal	8,000	M	I	I	I	I	I	I	C	A
15	selective_null	8,000	M	C	C	C	C	I	C	C	A
16	bounded_reversal	8,000	P	I	I	I	I	I	I	C	E
17	bounded_reversal_n400	4,000	P	I	I	I	I	I	I	I	E

A.4 Diagnostic outcomes and stopping interpretation

Other scenarios expose the practical-versus-sign target mismatch: practical FDR is 0.39500 at the exact practical boundary and approximately 0.73316 in the medium-reversal scenario. Full scenario results are retained rather than summarized as a single global error rate.

These findings also have limits. Assumption-aligned Gaussian procedures frequently remain *inconclusive* under the stringent simultaneous Monte Carlo screen; that is not evidence that they are miscalibrated. Conservative bounded Hoeffding–Holm comparisons have zero observed false discoveries and 100% observed diagnostic coverage, but practical power is zero at 40 windows and approximately 0.05263 at 400. The campaign does not certify a generally useful replacement procedure. Of 144 original method/scenario classifications, 45 indicate concern, 74 are inconclusive, 9 meet

the empirical tolerance screen, and 16 abstain outside their domain. Seven of the nine compatible entries arise in the degenerate exact-ties scenario; the remaining two are bounded Hoeffding.

Completion and precision must also be separated. Scenarios 3 and 17 stop at 2,000 and 4,000 studies; the other sixteen finish at 8,000. Six scenarios meet the original precision objective, while twelve stop at the maximum count with precision gaps. The original eight-method uncertainty family controls stopping. The expanded 17-method report has its own, wider allocation; additions do not retrospectively become prespecified. Neither maximum-count completion nor an inconclusive calibration screen is a successful calibration claim.

A.5 Numerical boundary case and substantive limitations

The saved scenario 11, replicate 6628, comparison index 47 has a screened family of 43 entries. At rank 43 the stored p-value is 0.05. The original port evaluates $0.05 \times 43/43$ as 0.049999999999999996 and does not reject; the actual source evaluates $(43/43) \times 0.05$ as 0.05 and rejects. The mathematical bootstrap-grid comparison is $1/20 \leq 1/20$ and rejects. The old referee still reproduces its mismatch on this preserved record; the recovery contracts explicitly distinguish all three identities.

The reuse audit found 31 port-versus-source differences among the original 112,275 studies. At final aggregation, the largest difference in endpoint means between those implementations is 0.00025. Only the exact screened-BH addition changes final endpoint means relative to its original counterpart, with maximum change 8.56×10^{-7}; the other seven exact variants' endpoint means match their originals. Aggregate equality does not establish identity of every rejection vector.

These small arithmetic differences do not repair the larger statistical problems. Full-family historical BH has practical FDR 0.51875 in the block null and 0.778625 in the AR null. Grouped-bootstrap BY has ordinary coverage 0.88775 in the IID equal null and 0.759875 in the AR null. Under AR, cluster-t coverage is 0.8600 in the null and 0.864375 in the reversal scenario because fixed five-window clusters remain dependent. Conversely, low FDR can coexist with negligible power: in the block-reversal scenario, cluster-t practical power is approximately 0.00000658, oracle-normal 0.000296, and grouped-bootstrap zero. Bounded Hoeffding power is zero at $n = 40$ and 0.052632 at $n = 400$, with original-family bounds approximately $[0, 0.00401]$ and $[0.04462, 0.06065]$. Inconclusiveness, abstention, and conservatism remain findings rather than reasons to select a new reporting threshold.

B Evidence Identities and Reproduction Scope

The C01 full run is `c01-full-167b9074e5711420`; the recovery run is `stat01-recovery-v2-061933a64e27da20`. Execution commits are `2d8342e699f35044044ac518c0b75a3c4a1c768c` and `1ac4c8dd1421aabad05e6ce243cb911a6ed6c8e1`, respectively. The independent evidence check is pinned at commit `6b2eedd7aa254986b85c1bb901587641cbeac8b0`; the supporting analysis at `d22c7751af9c563d5a4d8a304c007f26112c4d12`. The canonical frozen-window hash begins `0d1aa06ccbee`; the physical window-file hash begins `97fbaf6a4b9b`; absolute C01 metrics begin `5700c4bbd738`. Full hashes, source paths, and stored-copy hashes are supplied in `evidence/input-hashes.json` and the input manifest.

The completed C01 campaign performed exhaustive event verification during execution. The final review separately checked all 5,760 terminal records and recomputed their metrics, all 164 launch-file fingerprints, all 144 reused identities, and all 720 request-condition records. Its fresh event replay covered 24 new-cell journals, structurally chosen from the first nonpilot window per source at HIGH_PRESSURE, both capacities, and all adaptations. Independent report

checking covers 5,760 absolute rows, 15,840 rankings, 95,040 matched differences, and 26,136 reversal groups. The reproduction scripts regenerate displays only from those saved summaries and verifies their fingerprints.

STAT01's recovery validated all 5,360 batches during execution. The final review checked the complete batch inventory and independently recombined all final endpoint moments with high-precision arithmetic; its fresh record-level check covered 200 saved studies across eight scenarios, including the original failing record. It did not claim to rerun all raw studies. Recovery preserves original bytes and labels post-diagnosis comparisons separately. Full raw-record validation and display regeneration are different reproducibility tasks. The manuscript packet supplies the latter, and the separate local candidate contains the complete saved raw records with verification recipes; neither workflow generates new outcomes.

The historical evidence remains under `paper/tompecs-m2`, including its statistical audit, claim changes, cross-metric set comparisons, and finite-reference ranking-recovery analyses. Those materials remain archived support rather than additional independent experiments. Related headroom and module-intervention projects share some infrastructure and topics, but their causal or native-engine results are not used as evidence for this paper. Historical windows, input reconstruction, and the archived 36/990 rule result are reused explicitly; C01 has separate execution semantics and STAT01 separate synthetic truth. Exact overlap with some older cross-project BurstGPT selections remains unresolved. This uncertainty limits claims of independent evidence across projects, not the recorded within-C01 nonoverlap check.

C Chronological Summaries

Figure 4 reports SLO-minus-chunked ANWG differences in four consecutive ten-window blocks. Across settings and capacities, block means range from −0.0120 to +0.2030 for Azure, −0.0300 to +0.2330 for Bailian, and +0.0150 to +0.3275 for BurstGPT. Negative differences occur in 8/48 Azure and 24/48 Bailian block/setting/capacity combinations, and none of the 48 BurstGPT combinations. The frozen EARLY/MIDDLE/LATE summaries show the same qualitative contrast: negative differences in 6/36, 20/36, and 0/36 combinations, respectively. These are corpus-order sensitivities, not estimates of a temporal process or guarantees for future traffic.

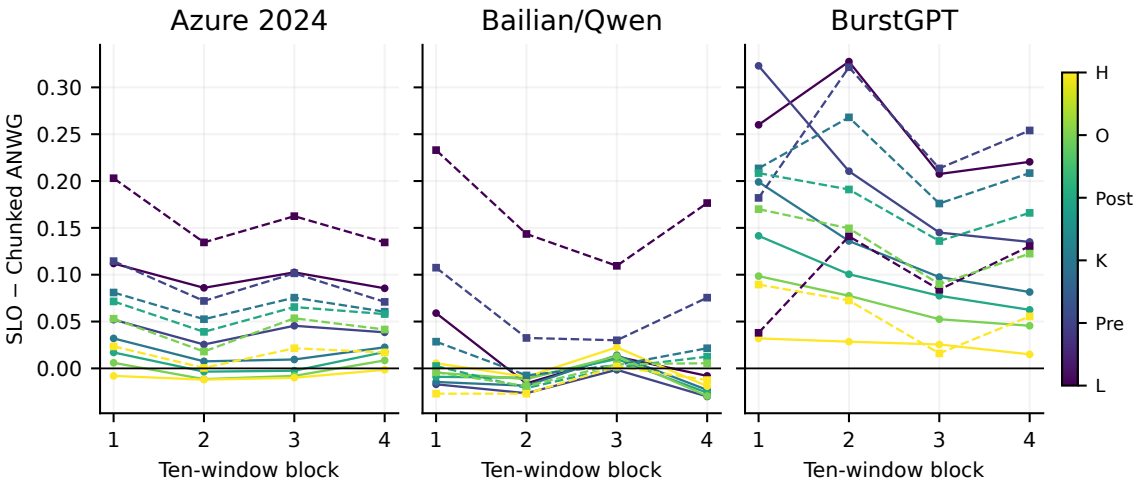


Fig. 4. Chronological block means of SLO-gated minus chunked-FCFS ANWG. Each block contains ten frozen windows. Colors identify inherited settings (L through H); solid circles and dashed squares denote four and eight slots. Blocks have unequal durations and are not independent temporal samples. The figure uses all 144 block/setting/capacity combinations.